

10 Lessons Learned from the Overwatch Agentic Harness Implementation

Repository: [ivenkwan/Overwatch](#)

Domain: AML (Anti-Money Laundering) Platform — Unified TradFi and Web3 Fund Flow Analysis Engine

Analysis Date: June 2026

Overview

The [ivenkwan/Overwatch](#) repository is a production-grade AML platform that serves as an exemplar of how modern enterprise systems are built using an **agentic harness approach** — where autonomous, specialized agents (services, pipelines, and workflow engines) collaborate through well-defined interfaces to accomplish complex, multi-step goals without human intervention at each stage. The repository spans a full N-Tier architecture: a Next.js frontend, a Python/FastAPI backend, a PostgreSQL + Apache AGE graph database, a Dagster ETL pipeline, a Flowable BPMN workflow engine, and Keycloak for identity management. The repository also integrates graphify, a code intelligence tool that generated a 201-node knowledge graph of the codebase, providing an additional layer of machine-readable architectural metadata.

The ten lessons below are extracted directly from real design decisions, code patterns, and even documented bugs and trade-offs found across the codebase. They are intended to guide developers in building their own agentic systems.

Lesson 1: Begin with a Spec-First, Harness-Second Mindset

The principle: Before writing a single line of agent code, produce a formal specification that the harness will enforce. The specification is the contract between the business domain and the agentic system.

What the code shows: The repository contains a dedicated [AML_spec.md](#) file that defines functional requirements (T+1 batch processing, circular flow detection, smurfing rules, money mule detection), non-functional requirements (30-second graph query timeout, super-node exclusion, PII masking), and an explicit RBAC role hierarchy (Viewer, Reviewer, Senior Investigator, Admin). Downstream code strictly implements this contract — the `pii_service.py` enforces data privacy rules, and the `rule_engine.py` executes exactly the typologies named in the spec.

The lesson for developers: An agentic harness without a spec becomes an ad-hoc script collection. The spec transforms vague goals like "detect money laundering" into machine-executable contracts. Write the spec as a markdown document first, then build agents that satisfy it — and use the `todo.md` pattern to track spec-to-implementation fidelity.

Lesson 2: Design Your Harness Around an Explicit Orchestration Layer

The principle: A true agentic harness requires a dedicated orchestration layer that manages agent lifecycle, sequencing, and state transitions — not just direct function calls.

What the code shows: Overwatch employs two complementary orchestrators. The **Dagster ETL pipeline** (`etl/daily_pipeline.py`) defines a declarative `@job` called `daily_update_job` that chains five `@op` agents in a strict dependency graph: `check_file_availability` → `load_csv_to_postgres` → `verify_data_load` → `update_graph_model` → `rename_to_ok`. Each `op` receives its input from the previous one, creating a typed data handoff chain. The **Flowable BPMN engine** (`aml_case_workflow.bpmn20.xml`) provides a separate orchestrator for the human-in-the-loop workflow, defining a `makerTask` → `checkerTask` → `approvalGateway` sequence that governs AML case investigation approvals.

The lesson for developers: Use purpose-fit orchestrators for different agent types. Batch/data agents are best orchestrated by workflow schedulers like Dagster or Airflow; human-approval workflows need BPMN engines like Flowable. Resist the urge to hardcode sequential calls — externalise the control flow so it can be inspected, audited, and modified independently of the agent logic.

Lesson 3: Idempotency Is Non-Negotiable for Reliable Agents

The principle: Every agentic operation that persists state must be idempotent — safe to re-run without side effects.

What the code shows: The ETL pipeline demonstrates textbook idempotency. In `load_csv_to_postgres`, each transaction is assigned a deterministic `txn_hash` computed as a SHA-256 of `customer_num` + `txn_date` + `txn_ref_no` + `txn_currency_amount`. All three `INSERT` statements — for customers, counterparties, and transactions — use `ON CONFLICT ... DO NOTHING`. The same pattern appears in `admin.py` for user provisioning: `INSERT INTO app.app_users ... ON CONFLICT (keycloak_user_id) DO UPDATE SET email = EXCLUDED.email`. The graph model updates use Cypher `MERGE` instead of `CREATE` to avoid duplicate nodes and edges.

The lesson for developers: Design agents assuming they will be re-run — due to failures, retries, or scheduled re-execution. Build idempotency into every write operation using deterministic natural keys, content hashes, and database `ON CONFLICT` clauses. In graph databases, always use `MERGE` instead of `CREATE`. An agent that can be safely retried without side effects is dramatically easier to operate in production.

Lesson 4: Use a Dead Letter Queue Pattern to Prevent Silent Data Loss

The principle: Agents that process external data must handle partial failures gracefully — errors in individual records must not silently discard data or crash the pipeline.

What the code shows: The Dagster pipeline implements a file-rename convention as its DLQ mechanism: the final `rename_to_ok` `op` renames the source file to `{filename}.OK` only after all upstream `ops` have completed successfully. If any `op` fails (e.g., `verify_data_load` raises a

Failure when count returns zero), the rename never executes, leaving the file available for reprocessing. The README explicitly calls out a "Dead Letter Queue (DLQ) for failed data ingestion triage" as a named operational feature. At the row level, the `load_csv_to_postgres` op drops null `txn_ref_no` rows with `.drop_nulls(subset=["txn_ref_no"])` before ingestion rather than crashing on them.

The lesson for developers: Build two tiers of DLQ into every agentic pipeline. At the **batch level**, use file markers, database flags, or queue acknowledgements to signal completion — never delete the source until the agent has fully succeeded. At the **row level**, filter bad data defensively before persistence and log rejected records to a separate error table for later inspection.

Lesson 5: Dual-Layer State Management — External Workflow Engine + Internal Database

The principle: In a compound agentic system, state must be tracked in two places simultaneously: in the workflow engine (for orchestration truth) and in the application database (for query truth). Neither alone is sufficient.

What the code shows: This is the most architecturally sophisticated lesson in the repository. The `cases.py` endpoint demonstrates the dual-write pattern explicitly: when a case action is performed, the code first calls `flowable_client.complete_task()` to advance the BPMN state machine, then separately executes `UPDATE app.cases SET status = $1` to sync the PostgreSQL record. When fetching a case, both stores are read: the `SELECT ... FROM app.cases` query fetches the relational state, while `flowable_client.get_active_task(workflow_instance_id)` fetches the live workflow task state from Flowable, and both are returned together as `case_info["activeTask"]`.

The lesson for developers: Workflow engine state and database state serve different masters. The workflow engine owns the control flow graph (what happened, in what sequence, and who is responsible next). The relational database owns the queryable, reportable state (what is the current status for dashboards, analytics, and API consumers). Build explicit synchronization points at every state transition.

Lesson 6: Compliance-First Design — Bake Audit, RBAC, and PII Masking into the Foundation

The principle: In regulated domains, compliance features are not add-ons. They must be woven into the data model, the API, and the database schema from day one — not retrofitted later.

What the code shows: Overwatch achieves this on three distinct levels:

- **Database-level RBAC:** The `01_rbac_initialization.sql` script creates two segregated DB roles — `aml_api_role` (cannot bypass RLS) and `aml_etl_role` (bypasses RLS for bulk loads). Row Level Security (RLS) policies enforce tenant isolation on `app.cases` and `app.alerts` using `current_setting('app.current_tenant', true)`.

- **Tamper-evident audit chain:** The `02_audit_tamper_evidence.sql` script creates a `trg_hash_audit_event` trigger that computes a SHA-256 hash chain on every INSERT into `app.audit_access_events`, linking each record's `previous_hash` to the prior row — making retroactive tampering cryptographically detectable. A second trigger (`trg_prevent_audit_tampering`) raises an exception on any UPDATE or DELETE, making the audit table append-only.
- **API-level PII masking:** The `pii_service.py` module recursively traverses all API response dictionaries and masks wallet addresses, account numbers, and personal identifiers based on the caller's role. Only `SENIOR_INVESTIGATOR` receives unmasked data, and every such access triggers `log_unmasking_event()` which writes to the audit trail.

The lesson for developers: Compliance in agentic systems is a cross-cutting concern. Implement it at the infrastructure level (DB triggers, RLS), the service level (PII masking middleware), and the API level (scope-enforced decorators) simultaneously. Never rely on a single enforcement point.

Lesson 7: Externalise Agent Configuration — Use the `.agents/` Convention

The principle: Agentic harnesses require a dedicated configuration layer that governs how the agent tooling navigates and understands the codebase. This configuration should be version-controlled alongside the code.

What the code shows: The repository introduces a `.agents/` directory at the project root — a convention for housing agent-readable rules and workflows. The `.agents/rules/graphify.md` file provides behavioural instructions to the coding agent: "Before answering architecture or codebase questions, read `graphify-out/GRAPH_REPORT.md` for god nodes and community structure" and "After modifying code files in this session, run `graphify update .` to keep the graph current." The `.agents/workflows/` directory further separates workflow-specific instructions from general rules. This externalises agent policy from code logic — changing the agent's behaviour requires only editing a markdown file, not modifying application code.

The lesson for developers: Adopt a `.agents/` directory in every agentic project. Separate **rules** (invariants that always apply) from **workflows** (task-specific procedures). Treat these files as first-class citizens in version control — they are the operating manual for your agent and should evolve as the system matures.

Lesson 8: Invest in Codebase Cartography — Build a Machine-Readable Graph of Your System

The principle: As an agentic codebase grows, agents need a structured, queryable map of the code — not just the raw files. A knowledge graph of the codebase becomes an essential navigation tool.

What the code shows: The `graphify-out/` directory contains the output of the `graphify` tool applied to the entire codebase. The `GRAPH_REPORT.md` shows that 201 nodes and 188 edges were extracted from 121 files (~169,902 words), resulting in 21 detected communities. The report identifies **God Nodes** — the most connected abstractions: `daily_update_job()` (6 edges),

`get_auth()` (5 edges), `mask_pii()` (5 edges), and `run_graph_promotion()` (5 edges) — which are the highest-risk components for refactoring. The report also surfaces **Knowledge Gaps**: 27 isolated nodes with ≤ 1 connection that may indicate undocumented components or missing interfaces.

The lesson for developers: Run a codebase knowledge graph tool (graphify, code2graph, or similar) as part of your CI pipeline and commit the output to version control. The "God Node" report tells you which modules are most fragile; the "Knowledge Gaps" report tells you where documentation and test coverage are weakest. Agent-assisted development is significantly more reliable when the agent can query a graph of your codebase rather than `grep` through flat files.

Lesson 9: Design for Hybrid Data Topology — Relational and Graph Together

The principle: Many real-world agentic tasks require both structured (relational) querying and graph traversal. Building an architecture that serves both from a single database avoids duplication and keeps agents' data access patterns consistent.

What the code shows: Overwatch's central architectural innovation is running Apache AGE (a graph extension) on top of PostgreSQL, providing both SQL and openCypher query capabilities against the same data store. The ETL pipeline performs the "dual-write": `load_csv_to_postgres` populates the relational `core.transactions` table, and `update_graph_model` immediately projects that data into the AGE graph using `MERGE` statements, creating Customer, Merchant, and Counterparty nodes and PAID/TRANSFERRED edges. The `session.py` establishes this dual mode on every database connection: `LOAD 'age'` and `SET search_path = ag_catalog, "$user", public`.

The lesson for developers: For investigative, fraud-detection, or network-analysis use cases, a pure relational or pure graph database is insufficient. Use a hybrid like PostgreSQL + Apache AGE (or equivalent). Design the ETL pipeline as the canonical "graph promotion" step — relational tables are the source of truth for operational data, while the graph is the analytical projection. Run both as a single, transactional unit.

Lesson 10: Embrace Incremental Honesty — Commit Your `todo.md` and Technical Debt

The principle: An agentic harness is never complete. Healthy development requires explicit tracking of what is implemented, what is deferred, and what is a known shortcut — in a form readable by both humans and agents.

What the code shows: The `todo.md` file is committed to the root of the repository and provides a granular `[DONE] / [ ]` checklist across frontend, backend, and fix categories. It honestly documents known technical debt: the comment in `cases.py` — "For now, generate a random tenant manually as we don't handle cross-schema perfectly here" — is a direct reflection of the `todo.md` item "Fix sync/async DB calls." The `auth.py` module acknowledges a default fallback: when no JWT token is present, it returns a hardcoded admin user, documented with the note "In production use env vars". The `SECURITY_SCAN_20260403.md` file tracks a security audit run against the repository.

The lesson for developers: A `todo.md` committed to the repository is not a sign of incompleteness — it is a sign of engineering maturity. It gives agents a persistent context window of known gaps, allows reviewers to distinguish MVP shortcuts from permanent design decisions, and prevents debt from going undocumented. Pair it with dated security scan reports to create a lightweight compliance audit trail that travels with the code.

Synthesis: The Agentic Harness Design Principles

The following table maps each lesson to the codebase artefact that demonstrates it most clearly:

#	Lesson	Key Artefact	Design Pattern
1	Spec-First Design	<code>AML_spec.md</code>	Contract-Driven Development
2	Explicit Orchestration	<code>daily_pipeline.py</code> , <code>flowable_client.py</code>	DAG + BPMN Orchestration
3	Idempotency	<code>load_csv_to_postgres()</code> , SHA-256 <code>txn_hash</code>	Deterministic Keys + ON CONFLICT
4	Dead Letter Queue	<code>rename_to_ok()</code> , <code>drop_nulls()</code>	File-marker DLQ + Row-level filtering
5	Dual-Layer State	<code>cases.py</code> <code>action_case()</code>	Dual-write: Flowable + PostgreSQL
6	Compliance-First	<code>01_rbac.sql</code> , <code>02_audit.sql</code> , <code>pii_service.py</code>	RLS + Hash Chain + Role-based Masking
7	Externalised Agent Config	<code>.agents/rules/graphify.md</code>	<code>.agents/</code> Convention
8	Codebase Cartography	<code>graphify-out/GRAPH_REPORT.md</code>	Knowledge Graph Navigation
9	Hybrid Data Topology	<code>session.py</code> , <code>update_graph_model()</code>	PostgreSQL + Apache AGE Dual- Mode
10	Incremental Honesty	<code>todo.md</code> , <code>SECURITY_SCAN_20260403.md</code>	Living Debt Tracker

Closing Observations

Two structural gaps are also worth noting as **anti-patterns to avoid**, visible in the repository's own `todo.md`:

- Incomplete frontend authentication** — the frontend does not yet attach JWT tokens to API calls, meaning RBAC enforcement only exists server-side. In an agentic system, every client — human or automated — must present credentials.
- Audit log persistence** — audit events are currently written to a logging framework rather than the tamper-evident database table. The `02_audit_tamper_evidence.sql` schema exists but the application-level `log_audit_event()` in `auth.py` writes to a Python logger, not to `app.audit_access_events`. This gap between the intended and actual audit sink illustrates

why the spec-to-implementation fidelity check (Lesson 1) must be continuous, not a one-time activity.

These gaps are not failures — they are the honest result of Lesson 10 in practice. The `todo.md` documents them explicitly, making the system inspectable and the path to production clear.